

继续第二章，主人。我们现在从 **HDFS 存储层** 进入 **MapReduce 计算层**。这一章非常适合考简答和伪代码题，尤其是 **WordCount**、**Shuffle**、**Combiner**、**Partitioner**、为什么不适合流式/在线/DAG。🌙

第2章 MapReduce | 开卷速查版

本章一句话： **MapReduce** 是一种面向大规模离线数据处理的分布式计算模型，把计算拆成 **Map** 和 **Reduce** 两类函数，系统负责数据划分、任务调度、Shuffle、容错和结果输出。

常考题型： 解释 MapReduce 思想；写 WordCount；解释 Shuffle / Combiner / Partitioner；说明 Map Task 和 Reduce Task 做什么；说明为什么 MR 适合离线批处理、不适合在线查询和流式计算。

翻页关键词： Map/Reduce抽象 → WordCount → key/value → Shuffle → Combiner → Partitioner → Map Task / Reduce Task → 容错 → 不擅长场景。

1. MapReduce 解决什么问题

MapReduce 来自 Google 的 MapReduce 论文，Hadoop MapReduce 受它启发实现。PPT里把它的特点概括为：**易于编程、扩展性好、高容错、适合 PB 级以上海量数据离线处理**。

它要解决的问题是：很多大数据任务本质上都可以拆成“局部处理 + 全局汇总”。比如统计 URL 访问频率、反向网页链接图、倒排索引等，PPT专门列出这些例子，用来说明要找到一种统一模型表达大量数据处理任务。

考试写法：

MapReduce 是一种大规模离线数据处理模型。它把计算抽象为 **Map** 和 **Reduce** 两个阶段，用户只需要编写这两个函数，系统负责数据划分、任务分发、**Shuffle**、结果收集和错误恢复等细节，因此适合 **PB 级海量数据** 的批处理任务。

2. Map 和 Reduce 到底是什么

Map 的作用是：对输入数据逐条处理，产生中间键值对。

Reduce 的作用是：把相同 key 的中间结果聚合起来，产生最终结果。

PPT里借 Lisp 的思想解释：Map 对每个元素独立计算，Reduce 对结果进行聚合。

标准抽象是：

```
Map(k1, v1) -> list(k2, v2)
Reduce(k2, list(v2)) -> list(k3, v3)
```

这里最关键的是 **k2**。它决定哪些数据会被分到同一个 Reduce 里。换句话说，MapReduce 题目里你第一步就要问自己：最后要按什么分组？这个东西通常就是中间 **key**。

考试写法：

Map 阶段负责从原始输入中提取中间键值对，**Reduce** 阶段负责对相同中间 **key** 的 **value** 列表进行聚合。中间 **key** 是 **Map** 和 **Reduce** 的连接点，决定了数据在 **Shuffle** 阶段如何分组。

3. WordCount 标准模板

WordCount 是 MapReduce 的必背题。PPT里给出的任务是：给定 TB/PB 级文件集合，统计所有文件中每个单词出现次数；Map 输入文件 id 和内容文本，输出 `<word, 1>`；Reduce 输入 `<word, [1,1,1,...]>`，输出 `<word, sum>`。

伪代码：

```
map(file_id, line):
    for word in split(line):
        emit(word, 1)

reduce(word, values):
    sum = 0
    for v in values:
        sum += v
    emit(word, sum)
```

数据流是：

```
原始文本
-> Map: <word, 1>
-> Shuffle: 按 word 分组
-> Reduce: <word, sum>
-> HDFS 输出
```

考试写法：

WordCount 中，Map 函数读取文本内容，将每个单词输出为 `<word, 1>`；Shuffle 阶段将相同 word 的记录聚集到同一个 Reduce；Reduce 函数对 value 列表求和，输出 `<word, count>`。

4. 一个 MR 作业的整体流程

一个 **MR Job** 包含 Map 和 Reduce 两个主要阶段。PPT明确说：Map 输入来自 HDFS，Reduce 输出到 HDFS，Map 输出经过 Shuffle 和格式变换后成为 Reduce 输入。

完整流程可以背成：

1. 从 HDFS 读取输入文件。
2. InputFormat 把输入切成多个 InputSplit。
3. 每个 Split 通常对应一个 Map Task。
4. Map Task 逐条读取记录，调用用户写的 map()。
5. Map 输出中间键值对 `<k2, v2>`。
6. 可选：Combiner 在 Map 端做本地预聚合。
7. Partitioner 决定每个 key 发给哪个 Reduce。
8. Shuffle 把相同 key 的数据拉到同一个 Reduce。

9. Reduce Task 对数据排序、分组，调用 `reduce()`。
10. Reduce 输出最终结果到 HDFS。

考试写法：

MapReduce 的执行流程是：输入文件先被划分为若干 **Split**，每个 **Split** 由一个 **Map Task** 处理。**Map** 输出中间键值对后，系统通过 **Partitioner** 和 **Shuffle** 将相同 **key** 的数据发送到同一个 **Reduce Task**。**Reduce** 对 **value** 列表进行聚合，最终结果写回 **HDFS**。

5. Split、Block、InputFormat、RecordReader

这个地方容易和 HDFS 混在一起。

Block 是 HDFS 的物理存储单位，比如 64MB 或 128MB。它属于存储层。

InputSplit 是 MapReduce 的逻辑输入单位。一个 Split 通常对应一个 Map Task。默认情况下，Split 经常和 HDFS Block 大小接近，但概念上不是同一个东西。

InputFormat 负责决定输入怎么切分、怎么读。比如文本文件按行读，表数据按行键读。

RecordReader 负责把 Split 里的数据解析成一条条 `<k1, v1>`，交给 map 函数。

可以这样记：

```
Block: HDFS 怎么存
Split: MapReduce 怎么算
RecordReader: 把 Split 读成一条条记录
```

考试写法：

Block 是 HDFS 的物理存储单位，**InputSplit** 是 MapReduce 的逻辑计算输入单位。**MapReduce** 通过 **InputFormat** 划分 **Split**，每个 **Split** 通常由一个 **Map Task** 处理，再由 **RecordReader** 将其中的数据解析成 **key/value** 记录交给 **map** 函数。

6. Shuffle 是什么

Shuffle 是 MapReduce 的核心，也是最容易考的概念。

它做的事情是：把 **Map** 输出的中间结果按照 **key** 重新分组，并传输给对应 **Reduce**。

比如 WordCount 中，所有 `<apple, 1>` 都必须送到同一个 Reduce，才能算出 apple 的总次数。Shuffle 就负责这个“按 key 聚合 + 网络传输 + 排序分组”。

简单流程：

```
Map1: <apple,1> <cat,1>
Map2: <apple,1> <dog,1>
```

```
Shuffle 后:  
ReduceA 收到 apple: [1,1]  
ReduceB 收到 cat: [1]  
ReduceC 收到 dog: [1]
```

考试写法:

Shuffle 是连接 **Map** 和 **Reduce** 的中间过程。它将 **Map** 输出的中间键值对按照 **key** 进行分区、传输、排序和分组，使得相同 **key** 的所有 **value** 都能被送到同一个 **Reduce Task** 处理。

7. Partitioner 是什么

Partitioner 决定 **Map** 输出的某个 **key** 应该发给哪个 **Reduce**。

默认策略通常是:

```
reduce_id = hash(key) mod R
```

其中 **R** 是 **Reduce Task** 的数量。

它解决的问题是: **Reduce** 有多个, 不能把所有数据都丢给同一个 **Reduce**, 所以需要按 **key** 分配。只要相同 **key** 算出来的 **reduce_id** 一样, 它们就会进入同一个 **Reduce**。

考试写法:

Partitioner 负责决定 **Map** 输出的中间 **key** 被发送到哪个 **Reduce Task**。默认方式通常是 **hash(key) mod R**, 其中 **R** 是 **Reduce** 数量。**Partitioner** 必须保证相同 **key** 被分配到同一个 **Reduce**, 这样 **Reduce** 才能完成正确聚合。

注意: **Partitioner** 可能导致 **数据倾斜**。如果某个 **key** 特别多, 比如热点词, 它对应的 **Reduce** 会特别慢。

8. Combiner 是什么

Combiner 可以理解成“**Map** 端的本地小 **Reduce**”。它在 **Map** 输出后、**Shuffle** 之前执行, 用来先在本地图聚合一部分数据, 减少网络传输。

WordCount 里适合用 **Combiner**:

```
Map 本地输出:  
<apple,1> <apple,1> <apple,1>  
  
Combiner 后:  
<apple,3>
```

这样 **Shuffle** 传输的数据就少了。

但是 Combiner 不能随便用。它适合 **sum**、**count**、**max**、**min** 这类满足结合律和交换律的操作。**average** 平均值不能直接用普通 **Combiner**，因为“局部平均值的平均值”不等于全局平均值。正确做法是传 **<sum, count>**，最后再除。

考试写法：

Combiner 是运行在 Map 端的本地聚合函数，用于在 **Shuffle** 之前减少中间结果数量和网络传输开销。它只适用于满足结合律和交换律的聚合操作，例如求和和计数；对于平均值这类操作，不能直接对局部平均值再次平均，需要传递 **sum** 和 **count**。

9. Map Task 和 Reduce Task 分别做什么

PPT中对 MR 1.0 的 Map Task / Reduce Task 讲得很直接：Map Task 解析每条数据记录，传递给用户写的 map 函数，并把 map 输出写入本地磁盘；Reduce Task 从 Map Task 远程读取输入数据，对数据排序，再按分组传给 reduce 函数。

Map Task：

```
读取 Split
-> RecordReader 解析记录
-> 调用 map()
-> 输出中间 <k2, v2>
-> 写本地磁盘
```

如果是 **map-only** 作业，也就是没有 Reduce，那么 Map 输出可以直接写 HDFS。

Reduce Task：

```
从多个 Map Task 拉取数据
-> 按 key 排序
-> 按 key 分组
-> 调用 reduce()
-> 输出到 HDFS
```

考试写法：

Map Task 负责读取输入 **Split**，解析记录并调用用户定义的 **map** 函数，将中间结果写入本地磁盘。**Reduce Task** 负责从各个 **Map Task** 拉取属于自己的分区数据，对数据排序和分组，然后调用 **reduce** 函数，最终把结果写入 **HDFS**。

10. MR 1.0 和 MR 2.0 架构口径

MR 1.0 的核心角色是 **JobTracker** 和 **TaskTracker**。

JobTracker 是主节点，负责接收作业、划分任务、调度 Map/Reduce Task、监控执行状态、处理失败。

TaskTracker 是工作节点，负责真正运行 Map Task 和 Reduce Task，并向 JobTracker 汇报状态。

考试里如果问 MR 1.0 架构，可以写：

Client 提交 Job

- > JobTracker 负责调度和监控
- > TaskTracker 运行 Map Task / Reduce Task
- > 任务完成后结果写 HDFS

MR 2.0 放到 YARN 上后，调度和资源管理拆开。容错里会出现 **MRAppMaster**，Task 会周期性向 MRAppMaster 汇报心跳。PPT里说 MRAppMaster 失败后由 YARN ResourceManager 重启；Map/Reduce Task 挂掉后，MRAppMaster 会重新申请资源并运行，Task 默认最多重试 4 次。

考试写法：

MR 1.0 采用 **JobTracker / TaskTracker** 架构，**JobTracker** 负责作业管理和任务调度，**TaskTracker** 负责执行具体的 Map/Reduce Task。**MR 2.0** 基于 YARN，将资源管理交给 **ResourceManager**，每个作业由 **MRAppMaster** 管理执行和容错。

11. MapReduce 的容错

MapReduce 的容错思想很简单：**任务失败就重新执行**。因为输入数据在 HDFS 上，MapReduce 计算又是确定性的，所以某个 Map Task 或 Reduce Task 挂了，可以在别的机器重新跑。

主要机制：

1. Task 周期性汇报心跳。
2. 管理节点发现 Task 失败或超时。
3. 为失败 Task 重新申请资源。
4. 在其他节点重新运行该 Task。
5. 如果重试次数超过上限，Job 失败。

Map 输出一般先写本地磁盘，所以如果某个 Map Task 所在机器挂掉，它的中间结果也可能丢失，需要重新运行这个 Map Task。Reduce 输出写到 HDFS，相对更可靠。

考试写法：

MapReduce 通过任务重试实现容错。Task 会周期性向管理组件汇报心跳，一旦 Task 失败或超时，系统会重新申请资源并重新执行该 Task。由于输入数据保存在 HDFS 中，Map 或 Reduce 的计算结果可以重新生成，因此可以容忍部分节点故障。

12. 数据本地性和推测执行

数据本地性 Data Locality：尽量把 Map Task 调度到数据所在节点上执行。这样不用把大数据通过网络搬到计算节点，减少网络传输。可以用一句话记：**计算去找数据，不让数据乱跑**。

推测执行 Speculative Execution：如果某个任务运行特别慢，系统可能在另一台机器上启动一个备份任务。两个任务谁先完成，就采用谁的结果，另一个取消。它解决的是“拖后腿机器”问题。

注意：推测执行不适合所有任务。如果任务会写外部数据库、发请求、产生副作用，重复执行可能出问题。

考试写法：

数据本地性指 MapReduce 尽量将 Map Task 调度到输入数据所在节点执行，以减少网络传输。推测执行指系统发现某个 Task 运行异常缓慢时，在其他节点启动备份任务，采用最先完成的结果，从而缓解慢节点对整体作业的影响。

13. MapReduce 不擅长什么

PPT明确列出 MapReduce 不擅长三类场景：**在线查询、流式计算、DAG计算**。在线查询要求像 MySQL 一样毫秒或秒级返回；流式计算的数据源会动态变化，而 MapReduce 输入数据集是静态的；DAG 计算中多个应用有依赖，后一个程序输入来自前一个输出。

展开写：

不适合在线查询：MapReduce 作业启动、调度、Shuffle、落盘开销大，没法毫秒级响应。

不适合流式计算：MapReduce 处理的是静态数据集，通常等一批数据准备好再跑。

不适合复杂 DAG / 迭代计算：多个 MR 作业串联时，中间结果常常写 HDFS，再被下一个作业读入，磁盘 I/O 很重。Spark PPT里也把 MapReduce 的性能问题归结为：中间结果写磁盘、多轮 MR 通过 HDFS 交换数据、调度开销大、没有充分利用内存计算、迭代计算有大量冗余 I/O。

考试写法：

MapReduce 适合离线批处理，但不适合在线查询、流式计算和复杂 DAG/迭代计算。原因是 MapReduce 输入通常是静态数据集，作业启动和调度开销较大，中间结果需要经过磁盘和 Shuffle，多轮作业之间还要通过 HDFS 交换数据，因此难以满足低延迟和连续处理需求。

14. 常见 MapReduce 设计题模板

做 MR 伪代码题，一定按这个顺序想：

1. 输入是什么？
2. 最终输出是什么？
3. 最终要按什么聚合？这个通常就是中间 key。
4. Map 输出什么 $\langle k2, v2 \rangle$ ？
5. Reduce 如何处理 $\text{list}(v2)$ ？

URL 访问频率统计

输入：<日志偏移量，日志行> 输出：<URL，total_count>

```
map(offset, line):
    url = extract_url(line)
    emit(url, 1)
```

```
reduce(url, counts):  
    emit(url, sum(counts))
```

倒排索引 Inverted Index

PPT给的输入输出是：输入 `<DocID, content>`，输出 `<word, list(DocID)>`。

```
map(doc_id, content):  
    for word in split(content):  
        emit(word, doc_id)  
  
reduce(word, doc_ids):  
    emit(word, unique(doc_ids))
```

反向 Web-Link 图

PPT给的输入输出是：输入 `<target, source>`，输出 `<target, list(source)>`。

```
map(target, source):  
    emit(target, source)  
  
reduce(target, sources):  
    emit(target, list(sources))
```

Distributed Grep

目标：从大量文件中找包含某个关键词的行。

```
map(file_id, line):  
    if keyword in line:  
        emit(file_id, line)  
  
reduce(file_id, lines):  
    for line in lines:  
        emit(file_id, line)
```

也可以是 map-only，不需要 Reduce。

本章高频简答模板

1. MapReduce 的基本思想是什么？ MapReduce 将大规模数据处理任务抽象为 Map 和 Reduce 两个阶段。Map 对输入数据逐条处理并输出中间键值对，Reduce 对相同 key 的 value 列表进行聚合。系统负责输入划分、任务调度、Shuffle、容错和结果输出，使用户可以较容易地编写大规模离线计算程序。

2. Shuffle 的作用是什么？ Shuffle 是 Map 和 Reduce 之间的数据重分布过程，负责将 Map 输出的中间键值对按照 key 分区、传输、排序和分组，使相同 key 的数据进入同一个 Reduce Task，从而完成后续聚合。

3. Combiner 有什么作用？使用条件是什么？ Combiner 在 Map 端对中间结果进行本地预聚合，可以减少 Shuffle 数据量和网络传输开销。它适合满足结合律和交换律的操作，如 sum、count、max、min；平均值不能直接对局部平均值再平均，需要传递 sum 和 count。

4. Partitioner 的作用是什么？ Partitioner 决定 Map 输出的 key 应该发送到哪个 Reduce Task。默认通常采用 $\text{hash}(\text{key}) \bmod R$ ，其中 R 是 Reduce 数量。它必须保证相同 key 被分配到同一个 Reduce，才能保证聚合结果正确。

5. 为什么 MapReduce 不适合流式计算？ MapReduce 的输入数据集通常是静态的，作业以批处理方式执行，需要经历任务启动、调度、Shuffle 和结果落盘等过程，不能对持续到来的数据进行毫秒级连续处理，因此不适合流式计算。

6. 为什么 MapReduce 不适合复杂 DAG / 迭代计算？ 复杂 DAG 或迭代计算往往需要多个 MR 作业串联，前一个作业的输出写入 HDFS，后一个作业再从 HDFS 读取，产生大量磁盘 I/O 和调度开销。MapReduce 缺少内存复用和统一 DAG 优化，因此效率较低。

本章易混点

Map 输出写哪里？ 一般写本地磁盘，供 Reduce 远程拉取；如果是 map-only 作业，可以直接写 HDFS。

Reduce 输出写哪里？ 最终结果写 HDFS。

Block 和 Split 一样吗？ 不一样。Block 是 HDFS 存储单位，Split 是 MapReduce 计算输入单位。默认可能大小接近，但概念不同。

Combiner 是必须的吗？ 不是。它是优化手段，能减少 Shuffle 数据量，但不能改变最终结果。

Shuffle 只是网络传输吗？ 不只是。它包括分区、传输、排序、分组等一整套过程。

MapReduce 高容错靠什么？ 靠 HDFS 保存输入数据，靠 Task 心跳检测和失败重试重新计算结果。

本章看到题目怎么写

看到“**解释 MapReduce**”：写 Map/Reduce 两阶段 + key/value + 系统隐藏分布式细节。

看到“**写 WordCount**”：Map $\langle \text{word}, 1 \rangle$ ，Shuffle 按 word 分组，Reduce 求和。

看到“**Shuffle 是什么**”：写按 key 分区、传输、排序、分组，把相同 key 送到同一个 Reduce。

看到“**Combiner**”：写 Map 端本地聚合，减少网络传输，要求结合律/交换律。

看到“**Partitioner**”：写 $\text{hash}(\text{key}) \bmod R$ ，保证相同 key 到同一个 Reduce。

看到“**为什么 MR 慢 / 不适合在线流式**”：写作业启动调度开销大、中间结果落盘、输入静态、Shuffle 重、低延迟差。

下一章就是 **HBase**，它的主线是：HDFS/MR 适合离线顺序处理，但不支持高并发随机读写，所以引出 BigTable/HBase。